

Country-Scale Offline Speed-Limit Inference on Smartphones: A Runtime Data Architecture Study

Anonymous Authors

Submission prepared for double-blind review

Abstract—This paper evaluates runtime data architectures for low-latency, offline-capable smartphone speed-limit inference using country-scale OpenStreetMap (OSM) data. We compare four storage/query scenarios across three query modes and a 30-day daily-diff update analysis using one common benchmark harness. Within this benchmark scope, a single-file embedded spatial database (S3) provides the strongest device-side latency and lower update overhead than the alternative embedded variant S4, while tiled packs (S2) improve substantially over the global-index baseline (S1). The result is an architecture recommendation for the current runtime-data layer, not a claim about end-to-end ISA effectiveness or safety impact.

Index Terms—Intelligent transportation systems, intelligent speed assistance, mobile GIS, OpenStreetMap, spatial indexing, offline-first

I. INTRODUCTION

Excessive speed remains a major crash-severity factor, and the deployment of Intelligent Speed Assistance (ISA) is now required in new EU vehicle types [1]–[3]. However, the installed vehicle base is substantially larger than annual new registrations, which motivates retrofit-capable software solutions that can run on smartphones with open geospatial data.

Most map-matching research focuses on inference models under sparse/noisy trajectories. In practical mobile deployment, a separate constraint is often dominant: country-scale map data must be queried with low latency under limited memory, storage I/O, and intermittent connectivity. This paper targets that runtime architecture problem.

The practical motivation is scale. EU passenger-car stock remains above 260 million vehicles, whereas annual registrations are only a fraction of that stock [4], [5]. In parallel, smartphone internet usage is high in both urban and rural contexts [6], making mobile offline-first assistance a feasible deployment channel for legacy vehicles.

The study is positioned for ITS tracks focused on infrastructure technologies and traffic data systems. The contribution is not a new probabilistic matching model; it is a deployment-oriented comparison of runtime data representations and update behavior for speed-limit inference.

Contributions:

- 1) A controlled comparison of four country-scale runtime architectures (S1–S4) built from identical source data.
- 2) Joint latency evaluation for cloud-like host execution and physical smartphone execution across three query modes.

- 3) A 30-day daily-diff update analysis to compare maintenance effort and update locality.
- 4) A benchmark-scoped architecture recommendation balancing query latency and incremental update cost.

II. RELATED WORK

Map matching for transport applications has been widely studied, including geometric/topological methods [7] and probabilistic sequence models such as HMM-based approaches [8]–[11]. In practice, these sequence models are the standard mechanism for stabilizing trajectory inference: they favor staying on the current road unless a turn transition is sufficiently plausible under network connectivity and motion evidence [8], [10]–[12]. Later work further improved robustness for sparse trajectories and route-level inference [13]–[16].

This model structure is also reflected in production routing engines. OSRM, Valhalla (Meili), and Mapbox map matching expose candidate emission/observation controls, transition plausibility settings (including GPS uncertainty or search radii), and confidence-style outputs for downstream use [17]–[19]. To reduce online transition-check cost, pre-computed transition structures have also been proposed; FMM uses a UBODT-style table so route-distance checks become lightweight lookups during matching [20].

This paper complements that line of work by isolating the systems layer around map data packaging and spatial access. R-tree indexing [21] is treated as an established mechanism whose deployment characteristics are compared across architecture variants.

Compared with prior ITS map-matching studies, the design focus here is on country-scale runtime packaging and update operability, not only per-query geometric accuracy. This positioning is relevant to production constraints in connected and non-connected operation: an architecture that is fast in a server benchmark may fail under mobile sandbox I/O constraints, while a compact architecture can preserve both latency and maintainability.

III. TECHNICAL VISION

The technical vision for our research project is an offline-first ISA retrofit platform that can scale from map consumption to crowd-supported map maintenance without requiring user accounts. The initial runtime knowledge base is built from OpenStreetMap and regulatory datasets, then released as immutable artifacts for cloud distribution and smartphone-local execution. On device, location and heading signals are

TABLE I
COMBINED DATASET PROFILE AND SPEED-LIMIT PROVENANCE
(GERMANY SNAPSHOT).

Metric	Value
Input file size	4.70 GB
Car-drivable ways	7,963,481
maxspeed=* ways	2,637,020
zone:maxspeed=* ways	172,116
Explicit speed-limit ways	2,639,693
Area-context objects	129,459
Explicit tagged limit (share)	33.15%
Implicit regulatory tag (share)	0.98%
Default-rule fallback (share)	65.87%

processed by a low-latency inference engine against the local baseline runtime database to produce speed-limit context under intermittent or absent connectivity.

The long-term extension is a local-first observation loop. Drivers can contribute potential speed-sign updates through voice input and on-device computer-vision capture when road view is available. Observations are normalized locally, stored with pseudonymous device-ID context, and queued for deferred synchronization. The backend then applies spatial and rule validation, resolves conflicts, and merges accepted updates into the global speed intelligence store, with an optional moderated export path to external map ecosystems.

This creates two coordinated loops: a baseline update loop from authoritative sources and an observation loop from user devices. Figure 1 highlights the paper scope in blue. The methodology in the next section is intentionally restricted to that scope: country-scale baseline preparation, runtime data packaging scenarios, and on-device inference query behavior. The lower sync-observation stack is architecturally specified but not benchmarked in this manuscript.

IV. METHODOLOGY

A. Data scope and regulatory baseline

All scenarios use the same OSM Germany snapshot (`germany-latest.osm.pbf`, 4.70 GB) [22]. Preprocessing retains car-drivable `highway=*` classes and associated speed annotations. Table I combines dataset profile statistics with speed-limit provenance after rule-aware fusion [23], [24].

When explicit speed tags are absent, the legal default baseline for Germany (50 km/h urban, 100 km/h rural) is applied [25]. This rule-aware fallback is necessary because most ways do not carry explicit speed tags. In the benchmark harness, the urban-versus-rural signal is approximated from built-up residential polygon containment with place-based fallback when no enclosing residential polygon is available; administrative polygons are retrieved separately for city labels and traceability. The inference order is: user input (if present), then way-level `maxspeed` retrieval through `bbox/hybrid/polyline`, and finally legal-rule fallback, which requires the area-context query measured here as `polycontainment`.

B. Runtime architecture scenarios

The four scenarios reflect distinct deployment trade-offs. S1 is a global-index baseline that queries country-scale index files directly. Its implementation is simple, but each lookup can trigger broader scans and higher I/O pressure. S2 introduces explicit spatial partitioning with content-addressed tiles. This localizes reads to relevant map partitions and supports selective tile-level refresh when updates arrive. S3 shifts to a single-file embedded database with spatial indexing; query execution performs direct spatial-window retrieval followed by geometric ranking, reducing candidate fan-out and simplifying deployment to one artifact. S4 adds a tile-membership prefilter before spatial-index intersection, aiming to prune dense urban candidate sets at the cost of an additional filtering stage.

For deployment, S2–S4 use immutable file artifacts per build and are therefore suitable for object-storage distribution, stateless worker replication, and autonomous offline execution on mobile devices.

C. Query modes

All scenarios share the same ranking score:

$$\text{score} = d + \lambda_h \cdot \Delta\theta + p_u \quad (1)$$

where d is the geometry distance term, $\Delta\theta$ is bidirectional heading mismatch, λ_h is heading weight, and p_u is an optional unknown-highway penalty.

The three query scenarios are designed to cover practical operating modes for moving-vehicle inference:

- **bbox:** d is computed from haversine distance to the clamped point on each candidate bounding box. This is the fastest approximation and acts as a coarse candidate filter.
- **polyline:** d is computed as minimum point-to-segment distance over full candidate geometry. This provides the highest geometric fidelity, especially on curved and elongated segments.
- **hybrid:** candidates are first ranked by `bbox`; a limited top- N subset is then refined using `polyline`. This aims to preserve near-`polyline` quality while reducing compute cost.

These scenarios capture a practical latency-fidelity spectrum and are evaluated identically for all runtime architectures.

D. Update analysis protocol

Incremental behavior is evaluated over a 30-day daily-diff window (2026-01-22 to 2026-02-20). The analysis measures changed ways, speed-tag changes, and scenario-specific refresh effort proxies (partition invalidation for S1/S2; simulated patch runtime for S3/S4).

The update protocol uses daily replication deltas to approximate operational maintenance effort between full snapshot checkpoints. This allows comparing granular partition invalidation (S1/S2) with patch-like database updates (S3/S4) under the same observation horizon.

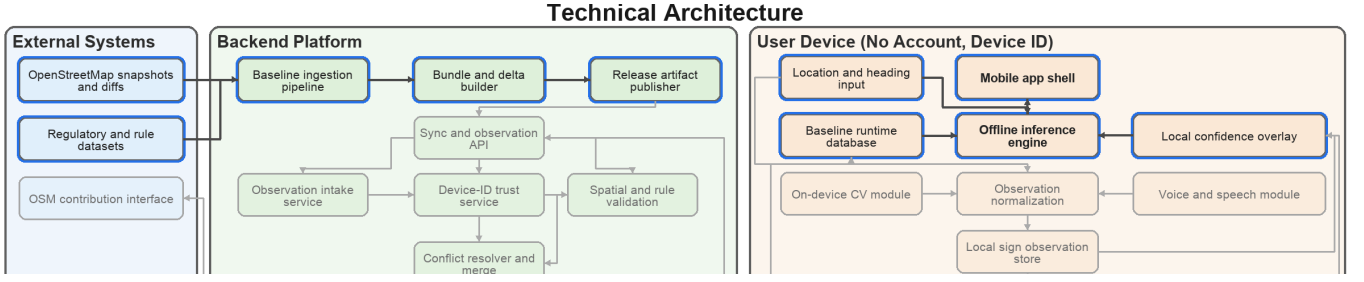


Fig. 1. Paper-scope technical architecture. Blue highlights indicate the part evaluated in this manuscript (top two rows, excluding the Sync and observation API).

TABLE II

MEAN LATENCY MATRIX (ms) AT THE BENCHMARK COORDINATE.

Mode	S1	S2	S3	S4
Online (cloud-like)				
bbox	5451.24	172.21	64.74	166.26
hybrid	10152.12	169.25	29.56	64.00
polyline	10300.65	170.65	38.88	72.03
polycontainment	1.51	1.07	0.94	0.44
Offline (device)				
bbox	4526.75	232.58	43.77	792.13
hybrid	2935.52	61.03	24.10	99.57
polyline	3122.36	76.20	39.16	78.54
polycontainment	3.14	0.81	0.74	2.79

V. EXPERIMENTAL SETUP

Two deployment contexts are measured:

- **Cloud-like host:** MacBook Pro (Apple M4 Max, 48 GB RAM), macOS 15.5.
- **Offline device:** physical iPhone 14 Pro (iOS 26.2.1), data stored in mobile sandbox.

The benchmark uses a fixed Berlin probe point (52.5200, 13.4050), heading 90°, top- $k = 5$, and four measured query components (bbox, hybrid, polyline, polycontainment). The harness intentionally isolates storage/query architecture and therefore does not reproduce the full continuity-aware iPhone matcher; it measures candidate retrieval, geometric ranking, and area-context lookup under one common protocol.

Runtime footprint differs substantially by architecture: S1 uses 7 files (5.51 GB), S2 uses 109,333 files (5.10 GB), S3 uses 1 file (2.60 GB), and S4 uses 1 file (3.44 GB).

VI. RESULTS

A. Latency comparison

Table II reports mean latencies (ms) across deployment contexts and query modes, while Figure 2 presents the same matrix in scenario-first view.

S3 is the strongest overall device configuration for decision-relevant query modes (hybrid, polyline). Compared with S1, host-side speedups now reach $343.4\times$ (S3, hybrid). On device, the ordering remains $S3 < S2 < S4 \ll S1$ for hybrid and polyline.

An important observation is that S4 does not uniformly dominate S3. While S4 applies additional prefiltering intended

TABLE III

THEORETICAL QUERY-THROUGHPUT PROXY FROM WORST-CASE PER-FIX COMBINED LATENCY ($t_{\text{MAXSPEED}} + t_{\text{POLYCONTAINMENT}}$).

Scenario	Mean ms	Hz	Best v_{proxy}	Worst v_{proxy}
S1	3531.35	0.28	12.2	7.9
S2	124.08	8.06	582.1	154.2
S3	36.42	27.46	1449.3	808.8
S4	326.20	3.07	442.7	45.3

to reduce dense-area fan-out, the extra stage increases overhead in some device-side conditions, including the bbox mode where S4 regresses strongly versus S3. This indicates that multi-stage filtering needs workload-specific tuning and is not automatically beneficial.

For the benchmark harness’s area-context retrieval, the measured built-up classification stage is 3.14 ms (S1), 0.81 ms (S2), 0.74 ms (S3), and 2.79 ms (S4). At the Berlin benchmark probe, all scenarios resolve the point as *inside-city* under residential polygon containment; administrative polygon lookup remains a separate labeling step. The low millisecond-scale polycontainment latencies are expected because RTree prefiltering reduces candidate polygons to a very small set before exact ring containment checks.

To express combined way-query plus area-context latency as a simple query-throughput proxy under worst-case per-fix operation, the following relation is used:

$$f_{\text{cycle}} = \frac{1000}{t_{\text{maxspeed}} + t_{\text{polycontainment}}}, \quad v_{\text{proxy}} = 3.6 \cdot f_{\text{cycle}} \cdot \Delta s \quad (2)$$

where t_{maxspeed} is way-query latency, $t_{\text{polycontainment}}$ is polygon containment latency, and $\Delta s = 10\text{m}$ is the inter-fix spacing assumption. These values are theoretical microbenchmark proxies, not operational vehicle-speed limits.

B. Incremental update workload

Table IV summarizes daily-diff update effort, and Figure 3 shows daily dynamics for additions, removals, speed-tag changes, and administrative-boundary way changes.

S2 improves refresh locality versus S1 (fewer invalidated partitions), while S3 is more efficient than S4 among database-centered variants ($1.33\times$ lower mean patch runtime for maxspeed updates and $4.30\times$ lower mean patch runtime for polygon updates).

TABLE IV
30-DAY DAILY-DIFF UPDATE SUMMARY (2026-01-22 TO 2026-02-20).

Metric	Total	Mean/day
Changed ways (+, −, modified)	1,829,171	60,972
maxspeed-related tag changes	139,999	4,667
Administrative-boundary way changes (boundary=administrative, level 8/9)	1,493	49.8
Invalidated S1 grid cells	201,355	6,712
Invalidated S2 tiles	80,536	2,685
S3 simulated patch runtime	30 719.38 ms	1023.98 ms
S4 simulated patch runtime	40 710.02 ms	1357.00 ms
Polygon-update S1 invalidated grid cells	15,151	505.03
Polygon-update S2 invalidated tiles	3,507	116.9
Polygon-update S3 simulated patch runtime	87.38 ms	2.91 ms
Polygon-update S4 simulated patch runtime	375.82 ms	12.53 ms

TABLE V
INDICATIVE UPDATE TRANSFER TIMES UNDER MEASURED DAILY VOLUMES.

Scenario	Payload (MB/day)	Transfer time (s)
S1	340.64 MB	13.78–28.74
S2	132.25 MB	5.35–11.16
S3	5.08 MB	0.21–0.43
S4	6.78 MB	0.27–0.57

To estimate practical mobile transfer costs, payload transmission time is modeled as:

$$T = \frac{B_{\text{payload}} + B_{\text{overhead}}}{R} \quad (3)$$

with $B_{\text{overhead}} = 0.1 \cdot B_{\text{payload}}$. For European 5G conditions, R is bounded by reported ranges in recent network measurements [26], [27]. For scenario-wise comparability, the daily payload proxy combines maxspeed and polygon update workload; S1/S2 use partition-equivalent payload via median tile size, while S3/S4 use patch-byte proxy scaling from measured gzip patch size. Under this model, S3 remains the fastest update path, followed by S4, while S1/S2 transfer cost is dominated by broad partition refresh.

C. Deployment choice and track relevance

Under the present benchmark and its simple equal-weight treatment of latency and update proxies, S3 is the most practical default among the measured scenarios. The architecture provides low-latency on-device lookup, compact single-file deployment, and lower incremental update overhead than S4.

For ITSC positioning, this evidence primarily supports tracks in *Infrastructure and ITS Technologies* and *Traffic Data Analytics and Machine Learning*, with secondary relevance for *ITS Validation and Verification*.

VII. THREATS TO VALIDITY

The benchmark uses one fixed probe location and heading, and therefore does not represent all geometric or traffic contexts. The update analysis uses one 30-day window and should be extended with longer seasonal coverage. The harness also isolates storage/query architecture rather than the full

sequence-aware iPhone runtime. Finally, the current evaluation reports latency and update effort but does not yet include route-level accuracy or end-to-end driving-assistance effectiveness metrics in fleet-scale trials.

VIII. CONCLUSION

This paper evaluated four runtime data architectures for country-scale, offline-capable speed-limit inference on smartphones. Keeping source data and query logic constant isolates architecture effects: global indexes (S1) are not viable for low-latency mobile deployment, tiled packs (S2) improve locality, and embedded spatial databases dominate latency. Among embedded variants, S3 provides the best latency-update tradeoff in the present benchmark and is the most defensible default for the current runtime-data layer. The result supports an architecture choice, not a direct claim about end-to-end ISA effectiveness or safety impact.

Future work will extend the benchmark to route-level workloads, multi-country datasets, and longitudinal weekly checkpoints with intervening daily diffs.

IX. REPRODUCABILITY

All preprocessing, scenario construction, and benchmark steps are parameterized and versioned in the accompanying research codebase available at <https://github.com/>. The evaluation reported here is tied to the Germany snapshot dated 2026-02-23, with fixed probe location, heading, query modes, and scenario definitions stated in the methodology and evaluation sections.

REFERENCES

- [1] European Parliament and Council of the European Union, “Regulation (eu) 2019/2144 on type-approval requirements for motor vehicles and their trailers, and systems, components and separate technical units intended for such vehicles,” <https://eur-lex.europa.eu/eli/reg/2019/2144/oj>, 2019, accessed: 2026-02-23.
- [2] European Commission, “Commission delegated regulation (eu) 2021/1958 supplementing regulation (eu) 2019/2144 by laying down detailed rules concerning intelligent speed assistance and driver drowsiness and attention warning systems,” https://eur-lex.europa.eu/eli/reg_del/2021/1958/oj, 2021, accessed: 2026-02-23.
- [3] —, “New mandatory vehicle safety features from july 2024,” https://single-market-economy.ec.europa.eu/news/new-mandatory-vehicle-safety-features-2024-07-05_en, 2024, accessed: 2026-02-23.
- [4] Eurostat, “Passenger cars in the eu (statistics explained; data extracted in january 2026),” <https://ec.europa.eu/eurostat/statistics-explained/SEPDF/cache/25886.pdf>, 2026, accessed: 2026-02-23.
- [5] European Automobile Manufacturers’ Association (ACEA), “Car registrations: +0.8% in 2024; battery electric 13.6% market share in december,” <https://www.acea.auto/pc-registrations/new-car-registrations-0-8-in-2024-battery-electric-13-6-market-share-in-december/>, 2025, accessed: 2026-02-23.
- [6] Eurostat, “Internet access in cities and rural areas,” <https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20241122-1>, 2024, accessed: 2026-02-23.
- [7] M. A. Quddus, W. Y. Ochieng, and R. B. Noland, “Current map-matching algorithms for transport applications: State-of-the art and future research directions,” *Transportation Research Part C: Emerging Technologies*, vol. 15, no. 5, pp. 312–328, 2007. [Online]. Available: <https://doi.org/10.1016/j.trc.2007.05.002>

- [8] P. Newson and J. Krumm, “Hidden markov map matching through noise and sparseness,” in *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2009, pp. 336–343. [Online]. Available: <https://doi.org/10.1145/1653771.1653818>
- [9] Y. Lou, C. Zhang, Y. Zheng, X. Xie, W. Wang, and Y. Huang, “Map-matching for low-sampling-rate gps trajectories,” in *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2009, pp. 352–361. [Online]. Available: <https://doi.org/10.1145/1653771.1653820>
- [10] C. Y. Goh, J. Dauwels, N. Mitrovic, M. T. Asif, A. Oran, and P. Jaillet, “Online map-matching based on hidden markov model for real-time traffic sensing applications,” in *2012 15th International IEEE Conference on Intelligent Transportation Systems*, 2012, pp. 776–781. [Online]. Available: <https://doi.org/10.1109/ITSC.2012.6338627>
- [11] Z.-c. He, S. Xi-wei, L.-j. Zhuang, and P.-l. Nie, “On-line map-matching framework for floating car data with low sampling rate in urban road networks,” *IET Intelligent Transport Systems*, vol. 7, no. 4, pp. 404–414, 2013. [Online]. Available: <https://doi.org/10.1049/iet-its.2011.0226>
- [12] T. Hunter, P. Abbeel, and A. Bayen, “The path inference filter: Model-based low-latency map matching of probe vehicle data,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 15, no. 2, pp. 507–529, 2014. [Online]. Available: <https://doi.org/10.1109/TITS.2013.2282352>
- [13] M. Rahmani and H. N. Koutsopoulos, “Path inference from sparse floating car data for urban networks,” *Transportation Research Part C: Emerging Technologies*, vol. 30, pp. 41–54, 2013. [Online]. Available: <https://doi.org/10.1016/j.trc.2013.02.002>
- [14] M. Quddus and S. Washington, “Shortest path and vehicle trajectory aided map-matching for low frequency gps data,” *Transportation Research Part C: Emerging Technologies*, vol. 55, pp. 328–339, 2015. [Online]. Available: <https://doi.org/10.1016/j.trc.2015.02.017>
- [15] G. R. Jagadeesh and T. Srikanthan, “Online map-matching of noisy and sparse location data with hidden markov and route choice models,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 9, pp. 2423–2434, 2017. [Online]. Available: <https://doi.org/10.1109/TITS.2017.2647967>
- [16] M. Che, Y. Wang, C. Zhang, and X. Cao, “An enhanced hidden markov map matching model for floating car data,” *Sensors*, vol. 18, no. 6, p. 1758, 2018. [Online]. Available: <https://doi.org/10.3390/s18061758>
- [17] Project OSRM Contributors, “Osm api documentation: Match service,” <https://project-osrm.org/docs/v5.24.0/api/#match-service>, 2026, accessed: 2026-02-26.
- [18] Valhalla Contributors, “Valhalla meili documentation,” <https://valhalla.github.io/valhalla/meili/>, 2026, accessed: 2026-02-26.
- [19] Mapbox, “Map matching api,” <https://docs.mapbox.com/api/navigation/map-matching/>, 2026, accessed: 2026-02-26.
- [20] C. Yang and G. Gidófalvi, “Fast map matching, an algorithm integrating hidden markov model with precomputation,” *International Journal of Geographical Information Science*, vol. 32, no. 3, pp. 547–570, 2018. [Online]. Available: <https://doi.org/10.1080/13658816.2018.1474007>
- [21] A. Guttman, “R-trees: A dynamic index structure for spatial searching,” in *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, 1984, pp. 47–57. [Online]. Available: <https://doi.org/10.1145/602259.602266>
- [22] Geofabrik GmbH, “Geofabrik openstreetmap data extracts – germany latest,” <https://download.geofabrik.de/europe/germany-latest.osm.pbf>, 2026, accessed: 2026-02-23.
- [23] OpenStreetMap Wiki Contributors, “Key:highway,” <https://wiki.openstreetmap.org/wiki/Key:highway>, 2026, accessed: 2026-02-23.
- [24] —, “Key:maxspeed,” <https://wiki.openstreetmap.org/wiki/Key:maxspeed>, 2026, accessed: 2026-02-23.
- [25] Bundesministerium der Justiz, “Straßenverkehrs-ordnung (stvo), section 3 (geschwindigkeit),” https://www.gesetze-im-internet.de/stvo_2013/_3.html, 2013, accessed: 2026-02-23.
- [26] Opensignal, “Europe’s mobile network experience 2024,” <https://www.opensignal.com/reports/2024/08/europe-mobile-network-experience>, 2024, accessed: 2026-02-23.
- [27] European Commission, “Impact assessment report accompanying the white paper “how to master europe’s digital infrastructure needs?” (swd(2026) 13 final),” <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:52026SC0013>, 2026, accessed: 2026-02-23.

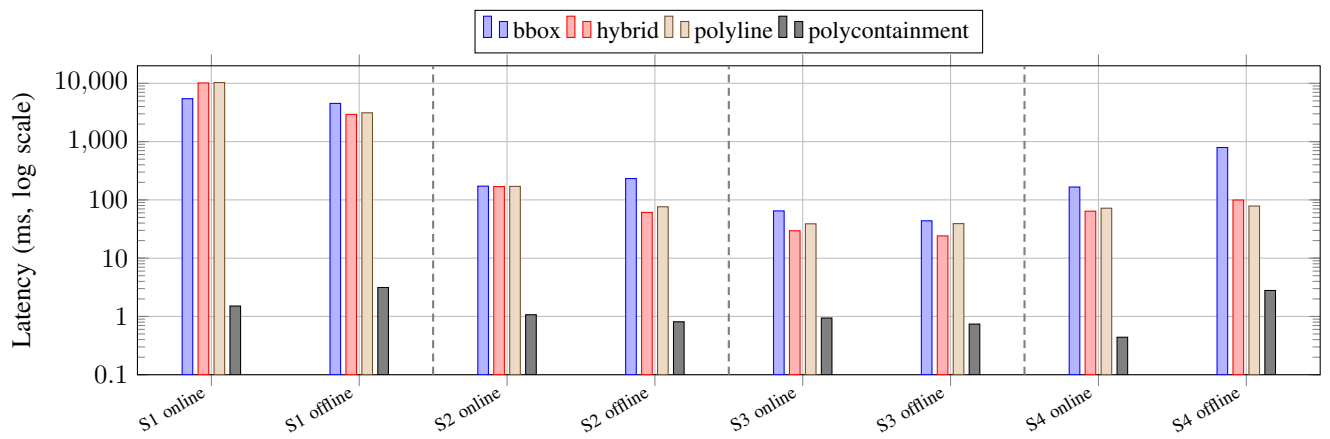


Fig. 2. Scenario-first latency view including polycontainment. For each scenario (S1–S4), online and offline columns are adjacent; dashed lines delimit scenario groups.

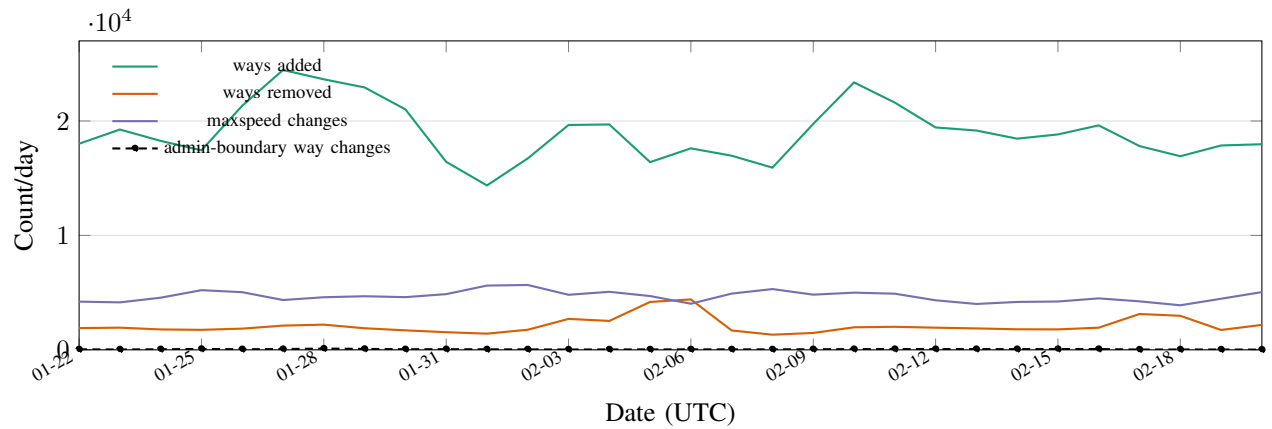


Fig. 3. Daily update dynamics over the 30-day analysis window, including administrative-boundary way changes.